Team members:

Sri Janani B (192421036)

Krithika M (192521385)

Prathiksha SN (192565090)

Shreya V (192565018)

Team1–Digital Banking Platform

Background Secure Bank Ltd. operates a nationwide digital banking platform serving more than five million customers. The bank offers online banking, mobile banking, digital payments, loan processing and investment services. To remain competitive with fintech companies, management mandates weekly software releases. The development team commits directly to the main branch, code reviews are optional, security testing is performed only before production deployment, third-party libraries are adopted without verification, password policies are weak, and API authentication is inconsistent. Application logs are generated but rarely reviewed. Developers have not received secure coding training. Recently, customers reported unauthorized login attempts and suspicious transactions. Internal investigation revealed exposed API keys, delayed security patches and outdated libraries.

Investigation Questions

1. Identify at least five security risks.

2. Identify at least five vulnerabilities and classify each as High, Medium or Low.

3. List possible cyber threats and explain their impact.

4. Prepare a Dev Ops Dev Sec Ops comparison table (minimum eight points).

5. Recommend five OWASP security best practices.

6. Explain the business impact if these issues remain unresolved.

7. Create a concept map linking Risk → Vulnerability → Dev Sec Ops Practice →Business Benefit.

8. Prepare a five-minute presentation summarizing your findings.

Case Study Analysis: Secure Bank Ltd. Digital Banking Platform

1. Five Key Security Risks

1. Risk of System-Wide Data Breach & Account Takeover (ATO): Inconsistent API authentication and weak user password policies leave customer accounts highly vulnerable to credential stuffing, brute-force attacks, and unauthorized session hijacking.
2. Risk of Supply Chain Compromise: Adopting third-party libraries without verification or automated Software Bill of Materials tracking exposes the platform to malicious dependencies, zero-day vulnerabilities, and backdoor injection.
3. Risk of Undetected Intrusion & Prolonged Dwell Time: Application logs are rarely reviewed or monitored with automated Security Information and Event Management tooling, allowing attackers to persist undetected inside the system.
4. Risk of Uncontrolled Source Code Contamination: Allowing developers to commit code directly to the main branch without mandatory peer reviews or branch protection rules creates a risk of malicious, buggy, or insecure code reaching production without oversight.
5. Risk of Insider Threat and Human Error: A complete lack of secure coding training for developers heightens the risk of introducing classic flaws (such as SQL Injection, XSS, or hardcoded credentials) into production code.

2. Identification and Classification of Vulnerabilities

S no	Vulnerability Identified	Severity Classification	Justification
1	Exposed API Keys & Secret Leaks	High	Exposed keys grant direct, unauthenticated access to backend banking services and sensitive databases.
2	Inconsistent API Authentication & Authorization	High	Weak or missing access controls on endpoints allow attackers to bypass login checks and access restricted financial transactions.
3	Outdated & Unvetted 3rd-Party Libraries	High	Third-party packages with known public vulnerabilities (CVEs) provide immediate, trivial exploit vectors for remote code execution.
4	Direct Commits to Main Branch without Mandatory Code Reviews	Medium	Lacks separation of duties and pull request checks, increasing the likelihood of unverified code deployment.
5	Weak Password Policies	Medium	Enables dictionary and brute-force attacks, leading to widespread unauthorized account access.
6	Unmonitored and Unanalyzed Application Logs	Medium	Impairs real-time incident detection, forensics, and rapid threat mitigation.
7	Lack of Secure Coding Training for Engineering Teams	Low	Directly causes repetitive human coding errors and anti-patterns throughout the software development life cycle.

3. Cyber Threats and Potential Impact

- Threat 1: Credential Stuffing & Account Takeover (ATO)
    - Mechanics: Attackers use automated tools to test leaked credentials against SecureBank's weak authentication endpoints.
    - Impact: Direct financial fraud, account drain, customer loss, and immediate loss of public trust.

- Threat 2: Software Supply Chain Attack
    - Mechanics: Malicious actors compromise upstream open-source packages or release compromised dependencies that SecureBank pulls into production without verification.
    - Impact: Complete infrastructure compromise, data exfiltration, and backdoored digital banking software.

- Threat 3: API Hijacking & Broken Object-Level Authorization (BOLA)
    - Mechanics: Attackers manipulate API calls with exposed keys or weak tokens to pull financial records or execute transactions belonging to other users.
    - Impact: Unprecedented exposure of Personally Identifiable Information (PII) and severe regulatory penalties.

- Threat 4: Man-in-the-Middle (MitM) and Session Hijacking
    - Mechanics: Inconsistent API security standards allow interception or replay of sensitive transit data between mobile devices and bank servers.
    - Impact: Interception of live financial transfers and session cookie theft.

4. DevOps vs. DevSecOps Comparison

Feature / Dimension	Traditional DevOps Environment	Shift-Left Dev Sec Ops Environment
1. Primary Objective	Rapid release velocity and deployment agility.	Rapid release velocity embedded with continuous security assurance.
2. Security Integration	Security is an isolated gate at the end of the pipeline.	Security is integrated directly into every stage of the CI/CD pipeline.
3. Code Testing	Functional and unit testing only; security tested pre-prod.	SAST, DAST, and SCA automated during pull requests and builds.
4. Dependency Management	Ad-hoc inclusion of third-party libraries.	Continuous SBOM generation and automated dependency scanning (Dependency-Check).
5. Branch Management	Direct commits to main without enforced review gates.	Enforced branch protection, feature branching, and mandatory PR approvals.
6. Credential Handling	Risk of hardcoded secrets or exposed API keys in repos.	Secret scanning (GitLeaks/TruffleHog) and dynamic vault integration.
7. Monitoring & Logging	Passive logging, rarely inspected until post-incident.	Centralized SIEM/SOAR with real-time log analysis and alert triggers.
8. Team Culture & Accountability	Security is exclusively the responsibility of the SecOps team.	Security is a shared responsibility across Development, Ops, and Security.

5.OWASP Security Best Practices Recommendations

1. Implement OWASP API Security Top 10 Safeguards: Standardize authentication across all APIs using OAuth 2.0 / OpenID Connect, strictly enforce granular object-level authorization (BOLA prevention), and utilize an API Gateway.

2. Automate Dependency & Supply Chain Management (OWASP SCA): Deploy automated Software Composition Analysis (SCA) tools (such as OWASP Dependency-Check or Snyk) in CI/CD pipelines to block builds containing known vulnerabilities.

3. Enforce OWASP Proactive Controls for Authentication & Passwords: Implement mandatory Multi-Factor Authentication (MFA), block leaked passwords, enforce strong length requirements, and rate-limit authentication endpoints.

4. Integrate Continuous Static & Dynamic Security Testing (SAST/DAST): Automate static code analysis (SAST) on every pull request and run dynamic scanning (DAST) on staging environments prior to release.

5. Establish Effective Logging and Monitoring (OWASP A09:2021): Centralize application logs, sanitize logs to prevent sensitive data leakage, and establish automated real-time alert thresholds for suspicious events.

6. Business Impact of Unresolved Issues

- Financial Loss & Direct Theft: Direct monetary exposure from fraudulent transactions, operational downtime, legal fees, and reimbursement costs to affected customers.
- Regulatory Non-Compliance & Heavy Fines: Non-compliance with banking mandates and privacy laws (e.g., PCI-DSS, GDPR) leading to massive regulatory fines and license suspensions.
- Catastrophic Brand & Reputational Damage: Loss of customer trust in a digital-first market leads to rapid churn to competing fintech platforms.
- Escalated Remediation Costs: Fixing vulnerabilities post-breach is exponentially more expensive than addressing flaws early in the development life cycle.

7. Concept Map: Linking Risk to Business Benefit

[RISK]

System-Wide Data Breach & Unapproved Production Code

[VULNERABILITY]

Direct Commits to Main Branch + Outdated 3rd-Party Libraries + Exposed API Keys

[DEVSECOPS PRACTICE]

Branch Protection Rules + Automated SAST/SCA in CI/CD + Secret Scanning

[BUSINESS BENEFIT]

Zero Downtime, Compliance Assurance, High Customer Trust & Uninterrupted Velocity

Detailed Traceability Matrix

- Chain 1:
    - Risk: Account Takeover & Transaction Fraud
    - Vulnerability: Inconsistent API Authentication & Weak Passwords
    - DevSecOps Practice: Enforce OAuth 2.0, API Gateway, and MFA Rate Limiting
    - Business Benefit: Fraud Reduction & Sustained Revenue Protection
- Chain 2:
    - Risk: Supply Chain Infection
    - Vulnerability: Unverified, Outdated 3rd-Party Libraries
    - DevSecOps Practice: Automated Dependency Scanning (SCA) & Policy Gates

- Business Benefit: Resilient Infrastructure & Reduced Remediation Expense.

8. Summary

Secure Bank Ltd. currently faces critical financial, legal, and operational risks driven by a development model that prioritizes rapid weekly delivery over basic security safeguards, leaving its five-million-customer platform exposed to severe vulnerabilities. Recent customer reports of unauthorized logins and suspicious transactions stem directly from continuous security flaws, including direct code commits to production without peer reviews, unvetted open-source dependencies, exposed API keys, inconsistent endpoint authentication, and unmonitored application logs. To mitigate these immediate threat vectors—specifically Account Takeover, Software Supply Chain Attacks, and API Hijacking—Secure Bank must transition from traditional DevOps to a Shift-Left Dev Sec Ops model. By integrating automated security scanners (SAST, DAST, and SCA) directly into the continuous integration and deployment (CI/CD) pipeline, enforcing strict branch protection rules, standardizing API security through OAuth 2.0, centralizing log analysis via SIEM tooling, and upskilling developers with secure coding training, the organization can remediate these systemic vulnerabilities without slowing down its innovation cadence. Ultimately, embedding security throughout the software development life cycle safeguards customer assets, preserves brand equity, ensures strict regulatory compliance, and guarantees that Secure Bank can scale securely in an increasingly competitive fintech environment.